9th Domain - Multimodal RAG
You are the career Gen AI coach, and you have great teaching skills & in-depth knowledge to help me achieve in-depth level knowledge for this particular domain. Teach me this end-to-end with proper explanation, examples, illustrations, etc. Make it super comprehensive. Below are the topics that you provided under this domain
Multimodal RAG
Document Understanding

Show more

Let's master Multimodal RAG. This domain is where most engineers top out — the moment documents stop being plain text, complexity multiplies fast. Knowing this deeply is a genuine differentiator.

The Core Problem: The World Isn't Text

Every RAG system we've studied so far assumes text is the input. But the real world is richer: insurance policy PDFs with embedded tables, product catalogs with images, earnings call videos, financial reports with charts, medical forms with handwriting. If your RAG system can only handle clean text, it fails the moment it encounters a scanned invoice or a slide deck.

Multimodal RAG is the engineering discipline of making retrieval work across modalities — and the interplay between them.

1. Document Understanding — The PDF Problem

PDFs are the most common document format in enterprise RAG, and they are the hardest to parse correctly. Understanding why requires a bit of PDF internals: a PDF is a rendering instruction set, not a semantic document. It says "place this character at coordinate (x, y) with this font". It doesn't say "this is a table header" or "this paragraph flows from that one". Extracting meaningful, structured text from a PDF is therefore non-trivial — and most naive extraction produces garbage for complex layouts.

Tool	Best for
Docling (IBM)	Open-source layout-aware parser (production-ready)
Azure Document Intelligence	Managed cloud OCR + layout analysis (handles scanned documents, checkboxes, key-value pairs)
PyMuPDF4LLM	Fast, lightweight, Markdown-friendly extraction
LlamaParse	Managed cloud API, complex layouts

Docling (IBM) — best open-source layout-aware parser (production-ready)

```
from docling.document_converter import DocumentConverter

converter = DocumentConverter()
result = converter.convert("insurance_policy.pdf")
markdown_text = result.document.export_to_markdown()

# Azure DI for scanned/handwritten docs
from langchain_community.document_loaders import AzureAIDocumentIntelligenceLoader

loader = AzureAIDocumentIntelligenceLoader(
    api_endpoint=os.environ["DI_ENDPOINT"],
    api_key=os.environ["DI_KEY"],
    file_path="scanned_claim.pdf",
    api_model="prebuilt-layout"  # or prebuilt-invoice, prebuilt-contract
)
docs = loader.load()
```

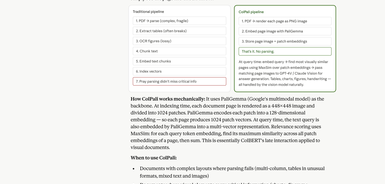
2. Image-Text RAG — Bridging Visual and Semantic Spaces

Once your documents contain images — product photos, charts, diagrams, figures — text-only RAG falls silently. An image of a chart showing quarterly premiums is invisible to a text retriever.

There are two fundamental approaches:

Approach 1: Convert images to text (captioning/description). Use a vision model (GPT-4V, Claude Vision, BLIP-2) to generate a text description of each image, then embed and retrieve that text. The advantage: standard text-retrieval pipeline works unchanged. The disadvantage: long — the description might miss details that matter for retrieval.

Approach 2: Embed images directly (multimodal embeddings). Use a joint embedding model (CLIP, BLIP-2, Nomic-embed-vision) that places both images and text into the same vector space. Query text is embedded, then compared directly to image embeddings. More lossless, but requires a multimodal vector DB or separate image index.



CLIP (Contrastive Language-Image Pre-training) is the foundational model for cross-modal retrieval. Trained by OpenAI on 400M image-text pairs with a contrastive objective: pull image and caption embeddings close, push non-matching pairs apart. The result: a shared embedding space where `embed_text("a photo of a cat")` is close to `embed_image(cat_photo)`. The VIT-L/14 variant is the production sweet spot: better quality than base, feasible to run on a single GPU.

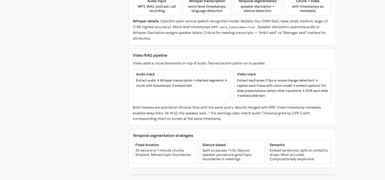
BLIP-2 takes it further with a bootstrapped approach: it bridges a frozen image encoder (ViT) with a frozen LLM (OPT or FlanT5) through a lightweight Q-Former module. This enables zero-shot image captioning AND visual question answering without full model fine-tuning. In RAG: generate detailed captions at indexing time, store both the caption and the image embedding.

Late fusion strategy: retrieve top-K from the text index and top-K from the image index independently, then combine results with RRF or weighted fusion before sending to the generator. The generator (GPT-4V, Claude Vision) then sees both text chunks and images in its context.

3. ColPali / Visual RAG — The Paradigm Shift

ColPali (Contextualized Late Interaction Patch Embeddings) is the most important recent development in document RAG. It makes a radical simplification: skip parsing entirely. Instead, treat each document page as an image, embed the page image, and retrieve by visual similarity.

The insight is powerful: why fight with complex PDF parsing, table extraction, OCR, and layout analysis — all of which are error-prone and expensive — when a vision model can simply see the page and understand it?



How ColPali works mechanically: It uses PaliGemma (Google's multimodal model) as the backbone. At indexing time, each document page is rendered as a 448×448 image and divided into 1024 patches. PaliGemma encodes each patch into a 128-dimensional embedding. The result: 1024 vectors per page, instead of one. At query time, the query is also embedded by PaliGemma into a set of vectors representing different attention scoring aspects. MaxSim for each query token: find the best match between similarity across all patch embeddings of a page, then sum. This is essentially ColBERT's late interaction applied to visual documents.

When to use ColPali:

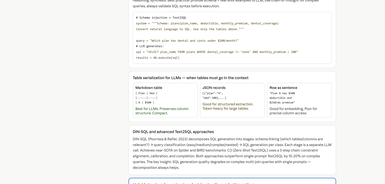
- Documents with complex layouts where parsing fails (multi-column, tables in unusual formats, mixed text and images)
- Documents where visual elements carry critical information (charts, diagrams, infographics)
- When you need fast indexing and can't afford parsing pipelines
- Scientific papers, annual reports, slide decks

When NOT to use ColPali:

- When you need to extract specific structured data (e.g., all deductible amounts for a database)
- Very long documents where rendering every page is expensive
- When downstream text search (BM25) is important — you lose that entirely
- When cost is a constraint (vision model inference at generation time)

Benchmark comparison: On DocVQA and similar document QA benchmarks, ColPali significantly outperforms traditional text-retrieval RAG, especially on pages with complex layouts and visual content. On plain-text document retrieval, it performs comparably or slightly worse.

4. Video & Audio RAG



5. Table & Structured Data — Text2SQL and Hybrid Retrieval

Tables are the place where RAG most often gets embarrassed in production. A table like:

Plan	Deductible	Premium/mo	Dental
Plan A	$500	$250	Full
Plan B	$300	$165	Basic

...contains precise, queryable data. A user asking "which plan has the lowest deductible?" needs the exact answer $300 (Plan B), not a vague semantic match. Vector similarity on serialized table text is fragile for precise data queries. SQL is the right tool.

Text2SQL — the right tool for precise structured queries

When to use exact comparisons, aggregations, filtering by precise values, ranking. When NOT: semantic questions, reasoning synthesis. Best practice: provide schema + few-shot examples to LLM, use chain-of-thought for complex queries, always validate SQL syntax before execution.

```
# Schema extraction + Text2SQL
sqlchain = \"\"\"Given: alexplan_name, deductible, monthly_premium, dental_coverage\n
Convert natural language to SQL. Use only the tables above.\n\n
query = "Which plan has dental and costs under $300/month?"
# LLM generates:
sql = "SELECT plan_name FROM plans WHERE dental_coverage = 'Yes' AND monthly_premium < 300"
results = db.execute(sql)
```

Table serialization for LLMs — when tables must go in the context

Markdown table	JSON records	Row as sentence

`| Plan | Ded |`
`|---|---|`
`| A | $500 |`
	`[{"plan":"A",`
	`"ded":500},...]`
		"Plan A has $500 deductible and Full dental coverage"

Best for LLMs. Preserves column structure. Compact.	Good for structured extraction. Token-heavy for large tables.	Good for embedding. Poor for precise lookup.

DIN-SQL and advanced Text2SQL approaches

DIN-SQL (Pourreza & Rafiei, 2023) decomposes SQL generation into stages: schema linking (which tables/columns are relevant?) → query classification (easy/medium/hard/extra-hard) → SQL generation per class. Each stage is a separate LLM call. Achieves near-SOTA on Spider and BIRD benchmarks. Cost: 2-3 calls. Worth it for complex queries (multi-table joins, subqueries, aggregations). Both approaches outperform single-prompt Text2SQL by 10-20% on complex queries. The key insight: SQL generation quality degrades on complex multi-join queries with single prompts — decomposition always helps.

Hybrid structured + unstructured retrieval — the production pattern

Most real queries need both. "Explain why Plan A's $500 deductible is higher than industry average" requires SQL for "$500 deductible, industry average $350" AND vector retrieval for "deductible rationale" from policy narrative. The routing logic: if query contains aggregation, comparison, or precise values → Text2SQL path. If query requires context, explanation, or "why" → vector retrieval path. If both → run both, merge results, let the generator synthesize. Implementation: route by keyword heuristics first (cheap), then by a synthesis node that combines structured query results with narrative context.

Interview Question Bank — Domain 9

Basics **Why is PDF parsing hard and which tool would you use for an insurance document corpus?**

PDFs are rendering instruction sets, not semantic documents — they store text as positioned glyphs with no structural metadata. A multi-column layout looks the reader feel positions a table has no cell boundaries to the binary format. For an insurance document corpus: IT documents are digital (born-PDF), use Docling — it uses deep learning layout detection to correctly handle columns, tables, and reading order, outputting clean markdown with table structure preserved. If documents include scanned pages (physical paper digitized), use Azure Document Intelligence — it has production-grade OCR + layout understanding trained on billions of forms and handles scanned quality. Never use PyMuPDF for complex insurance documents — it merges columns and destroys table structure.

Basics **What is CLIP and how does it enable image retrieval in RAG?**

CLIP (Contrastive Language-Image Pre-training) is an embedding model trained on 400M image-text pairs with a contrastive objective: make the embedding of an image and its caption close together, push non-matching pairs apart. The result is a shared embedding space where text and images are directly comparable. In RAG: at indexing time, pass each image through CLIP's image encoder to get a 512-dim vector. At query time, pass the text query through CLIP's text encoder. Run cosine similarity against the image index to find relevant images. The key property: you can query "a bar chart showing quarterly premiums" and retrieve an actual chart image without needing to extract text from it. ViT-L/14 is the production sweet spot — significantly better than base variants and feasible to run on a single GPU.

Mid-level **Explain ColPali and when you would use it over traditional text extraction RAG.**

ColPali treats document pages as images and embeds them using PaliGemma. Each page is rendered to a 448×448 image, divided into 1024 patches, each patch embedded into a 128-dim vector. Total: 1024 vectors per page (vs. 1 vector for a text chunk). At query time: embed the query into 128-dim vectors, then MaxSim: for each query token, find its maximum similarity across all page patches, sum to get a page relevance score. No parsing, no OCR, no table extraction — the vision model handles everything. Use ColPali when: complex document layouts that defeat text parsers, documents where visual elements (charts, diagrams) carry critical information, slide decks, financial reports, scientific papers. Use traditional text extraction when: simple text-heavy documents, cost-sensitive deployments (ColPali is 3-5x slower), RAGAS keyword search matters, long documents where rendering every page is expensive, or cost is constrained (ColPali requires GPU inference, more expensive than text).

Mid-level **When would you use Text2SQL vs vector retrieval for a question about a table?**

Text2SQL: when the question requires precise structured operations — exact comparisons ("which plan has deductible exactly $500"), aggregations ("average premium across all plans"), rankings ("cheapest plan with dental"), or filtering on precise values. The answer can be computed deterministically from the data. Vector retrieval: when the question requires semantic understanding of unstructured narrative content — "explain why Plan A has a higher deductible", "what does the premium calculation" "summarize the coverage philosophy across plans". The table doesn't contain this information; the policy narrative does. Hybrid: "explain why Plan A's $500 deductible is higher than industry average" — SQL for the numbers, vector for the explanatory context. In production, implement a classifier node that examines the query for aggregation terms, comparison operators, and precise values to route to SQL, fall through to vector otherwise, run parallel paths for hybrid queries.

Senior **Design a multimodal RAG pipeline for a corpus of 50,000 insurance policy PDFs that includes text, tables, charts, and some scanned historical documents.**

Multi-path ingestion by document type: (1) Digital PDFs (90% of corpus): Docling extraction → text chunks with markdown tables preserved → standard embed → Qdrant index. (2) Scanned PDFs (10%): Azure Document Intelligence → cleaned text + structured tables JSON → same index. Batch handling: extract all tables to a PostgreSQL database with plan_id as foreign key. Connect to Text2SQL path. Ingestion/retrieval: BLIP-2 captioning each extracted figure → embed captions → store in same Qdrant collection with metadata type=figure. Page-level ColPali index: render each page as image → ColPali embeddings → separate Qdrant collection. Query routing: LLM classifier routes to Langchain router: exact/aggregation → Text2SQL; semantic → Qdrant hybrid search (BM25 + dense, RRF fusion); visual/layout → ColPali page-level search; cross-modal → Qdrant hybrid search + ColPali page-level search. Metadata filtering: Qdrant payload filters on plan_id, doc_type, page_number, carrier_name, file_date. Cross-encoder reranker: top-20 candidates → top-5. Generation: gpt-4o-mini for simple queries, gpt-4o for complex queries. RAGAS on test queries, manual evaluation on edge-cases. Cost: ~$0.01/query average.

The Multimodal RAG Decision Tree

When you encounter a new document type or query pattern, reason through it this way:

What is the primary content of the document? Pure text → standard RAG. Text + tables → text RAG + SQL layer. Diagrams/charts embedded → CLIP indexing + captioning. Complex layout with visual elements → ColPali. Scanned/handwritten → Azure DI + text RAG. Videos/audio → Whisper + temporal segmentation.

What does the query need? Precise numbers, filtering, ranking → Text2SQL. Semantic understanding of narrative text → vector RAG. Visual content in a document → ColPali. Cross-modal → hybrid search + reranker.

Upgrade

Reply...

Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.